

DOCUMENTAÇÃO DO SISTEMA VISIONUP

Avaliação A2 — Análise, Modelagem, Desenvolvimento e Testes

Sistema web de cadastro e gerenciamento de candidatos

SUMÁRIO

1. INTRODUÇÃO

- 1.1 Objetivo do projeto
- 1.2 Escopo
- 1.3 Definições, siglas e abreviações

2. ANÁLISE DO PROBLEMA

- 2.1 Contexto do sistema
- 2.2 Stakeholders (partes interessadas)
- 2.3 Levantamento inicial

3. ANÁLISE DE REQUISITOS

- 3.1 Requisitos funcionais
- 3.2 Requisitos não funcionais
- 3.3 Regras de negócio
- 3.4 Casos de uso (descrição textual)

4. MODELAGEM DO SISTEMA

- 4.1 DFD (Diagrama de Fluxo de Dados)
- 4.2 Diagramas UML
 - 4.2.1 Diagrama de Casos de Uso
 - 4.2.2 Diagrama de Classes
 - 4.2.3 Diagrama de Sequência
 - 4.2.4 (Opcional) Diagrama de Atividades
 - 4.2.5 (Opcional) Diagrama de Estados

5. PROTOTIPAÇÃO

- 5.1 Protótipo no Figma
- 5.2 Descrição das telas (Documentação das interfaces)

6. ARQUITETURA DO SISTEMA

- 6.1 Visão geral da arquitetura
- 6.2 Tecnologias utilizadas
- 6.3 Diagrama de arquitetura

7. PROJETO DE BANCO DE DADOS

- 7.1 Modelo Conceitual
- 7.2 Modelo Lógico
- 7.3 Modelo Físico

8. DESENVOLVIMENTO / PROJETO FUNCIONAL

- 8.1 Código da aplicação (documentado)
- 8.2 Scripts SQL (documentados)
- 8.3 Padrões de codificação utilizados

9. PLANO DE TESTES

- 9.1 Estratégia de testes
- 9.2 Casos de teste
- 9.3 Resultados dos testes

10. CRONOGRAMA DO PROJETO

- 10.1 Etapas
- 10.2 Prazos
- 10.3 Marcos (milestones)

11. ORÇAMENTO

- 11.1 Custos estimados
- 11.2 Recursos necessários

12. IMPLANTAÇÃO E MANUTENÇÃO

- 12.1 Plano de implantação
- 12.2 Suporte e manutenção

13. REFERÊNCIAS BIBLIOGRÁFICAS

- 13.1 Referências

14. ANEXOS

- 14.1 Links úteis
- 14.2 Protótipos
- 14.3 Outros documentos

15. LINK DO REPOSITÓRIO

- 15.1 GitHub do projeto

Resumo do sistema

O VisionUp é uma aplicação web voltada ao cadastro e gerenciamento de candidatos. O sistema disponibiliza uma área para candidatos, na qual é possível realizar cadastro, autenticação, manutenção de dados pessoais e profissionais e envio de trabalhos e currículo. Também possui uma área administrativa para autenticação e consulta dos candidatos cadastrados, incluindo acesso aos arquivos enviados, visualização do LinkedIn e exclusão de registros.

A solução foi desenvolvida como projeto acadêmico utilizando PHP, MySQL/MariaDB, HTML, CSS, JavaScript e Bootstrap. O sistema também utiliza a API ViaCEP para preenchimento automático de dados de endereço a partir do CEP.

1. INTRODUÇÃO

1.1 Objetivo do projeto

Desenvolver uma aplicação web que centralize o cadastro de candidatos e permita a uma equipe administrativa consultar as informações e documentos enviados. O sistema busca reduzir o uso de formulários e armazenamento manual, organizando dados pessoais, endereço, contato, perfil profissional e arquivos em uma única solução.

1.2 Escopo

O escopo contempla a página inicial, autenticação de candidatos e administradores, cadastro de candidatos, consulta dos dados do candidato autenticado, atualização cadastral, upload de dois trabalhos e currículo, consulta administrativa dos candidatos, download de arquivos, acesso ao LinkedIn e exclusão de candidatos. O projeto não contempla um protótipo independente no Figma, conforme definido para esta documentação.

1.3 Definições, siglas e abreviações

PHP: linguagem utilizada no processamento do lado do servidor. HTML: linguagem de marcação das páginas. CSS: linguagem de estilos. JS/JavaScript: linguagem utilizada para comportamentos no navegador. SQL: linguagem de consulta e manipulação do banco de dados. API: interface de programação de aplicações. CEP: Código de Endereçamento Postal. CRUD: conjunto de operações de criação, consulta, atualização e exclusão. BD: banco de dados. PK: chave primária.

2. ANÁLISE DO PROBLEMA

2.1 Contexto do sistema

O processo de recebimento de candidaturas normalmente envolve coleta de dados pessoais, contatos, endereço, currículo e trabalhos. Sem um sistema centralizado, esses elementos podem ficar dispersos em arquivos e mensagens, dificultando a consulta pelos responsáveis. O VisionUp organiza esses dados em uma base estruturada e associa os documentos enviados ao registro do candidato.

2.2 Stakeholders (partes interessadas)

Candidato: pessoa responsável por criar e manter seu cadastro e enviar documentos. Administrador: usuário responsável por consultar os candidatos e seus materiais e excluir registros quando necessário. Equipe de desenvolvimento: responsável pela implementação, banco de dados, testes e manutenção do sistema. Responsável pela avaliação/projeto acadêmico: parte interessada na documentação, funcionamento e qualidade da solução.

2.3 Levantamento inicial

A partir do código fornecido, foram identificadas duas áreas de acesso. A página inicial direciona para Administradores ou Candidatos. O candidato pode se cadastrar, autenticar-se, visualizar e atualizar seus dados e enviar documentos. O administrador pode autenticar-se e consultar a lista de candidatos. A solução utiliza sessões PHP para manter o estado do usuário autenticado.

3. ANÁLISE DE REQUISITOS

3.1 Requisitos funcionais

RF01 — O sistema deve apresentar uma página inicial com acesso às áreas de administrador e candidato.
RF02 — O sistema deve permitir cadastro de candidato.
RF03 — O sistema deve verificar se o e-mail do candidato já está cadastrado.
RF04 — O sistema deve armazenar a senha do candidato com hash.
RF05 — O sistema deve permitir autenticação do candidato por e-mail e senha.
RF06 — O sistema deve manter sessão do candidato autenticado.
RF07 — O sistema deve consultar dados de endereço a partir do CEP por meio da API ViaCEP.
RF08 — O sistema deve permitir envio de dois trabalhos.
RF09 — O sistema deve permitir envio de currículo.
RF10 — O sistema deve armazenar os caminhos dos arquivos enviados.
RF11 — O candidato deve visualizar seus dados cadastrados.
RF12 — O candidato deve atualizar dados pessoais, contato, endereço e LinkedIn.
RF13 — O administrador deve autenticar-se.
RF14 — O administrador deve visualizar a lista de candidatos.
RF15 — O administrador deve baixar trabalhos e currículos.
RF16 — O administrador deve acessar o LinkedIn informado.
RF17 — O administrador deve excluir candidatos.
RF18 — O sistema deve permitir encerramento da sessão por logout.

3.2 Requisitos não funcionais

RNF01 — A interface deve ser responsiva para diferentes tamanhos de tela.
RNF02 — A aplicação deve utilizar componentes de interface do Bootstrap.
RNF03 — As operações de banco devem utilizar MySQL/MariaDB e a extensão mysqli.
RNF04 — Consultas sensíveis do módulo de candidatos devem utilizar prepared statements.
RNF05 — A senha do candidato deve ser armazenada com password_hash().
RNF06 — O sistema deve permitir upload de arquivos PDF, DOC e DOCX nos campos de trabalhos e currículo.
RNF07 — O sistema deve utilizar sessões PHP para controle do estado de autenticação.
RNF08 — O sistema deve possuir organização simples de arquivos, adequada ao contexto acadêmico.
RNF09 — O cadastro deve evitar duplicidade de e-mail para candidatos.
RNF10 — A solução deve apresentar mensagens de sucesso ou erro nas operações principais.

3.3 Regras de negócio

RN01 — O e-mail do candidato não pode ser cadastrado novamente.
RN02 — Nome, e-mail e senha são campos obrigatórios no processamento do cadastro.
RN03 — A senha do candidato é convertida para hash antes de ser gravada.
RN04 — O candidato autenticado é identificado pela sessão e pelo id_candidato.
RN05 — Os arquivos enviados são armazenados na pasta curriculos/.
RN06 — Os arquivos recebem nomes compostos por timestamp e identificadores adicionais para reduzir colisões.
RN07 — O administrador só consegue realizar as operações administrativas após autenticação no fluxo de login.
RN08 — O cadastro pode conter até dois trabalhos e um currículo.
RN09 — O endereço pode ser preenchido automaticamente a partir de um CEP válido.
RN10 — A exclusão de candidato remove o registro correspondente da tabela candidatos.

3.4 Casos de uso (descrição textual)

UC01 — Cadastrar candidato: o candidato informa seus dados, endereço, LinkedIn e documentos; o sistema valida campos básicos, verifica e-mail, gera hash da senha, move os arquivos e grava o registro.
UC02 — Autenticar candidato: o candidato informa e-mail e senha; o sistema consulta o registro, verifica a senha com password_verify() e cria a sessão.
UC03 — Consultar dados: após autenticação, o sistema exibe os dados pessoais, endereço, LinkedIn e documentos do candidato.
UC04 — Atualizar cadastro: o candidato altera dados pessoais, contato, endereço e LinkedIn; o sistema executa UPDATE pelo id_candidato da sessão.
UC05 — Autenticar administrador: o administrador informa nome e senha; o sistema consulta a tabela administrador e, em caso de sucesso, cria sessão administrativa.

UC06 — Consultar candidatos: o administrador acessa a lista e o sistema consulta a tabela candidatos.

UC07 — Baixar documentos: o administrador utiliza os links dos arquivos armazenados para baixar trabalhos e currículo.

UC08 — Excluir candidato: o administrador seleciona um candidato, confirma a ação e o sistema executa a exclusão pelo id_candidato.

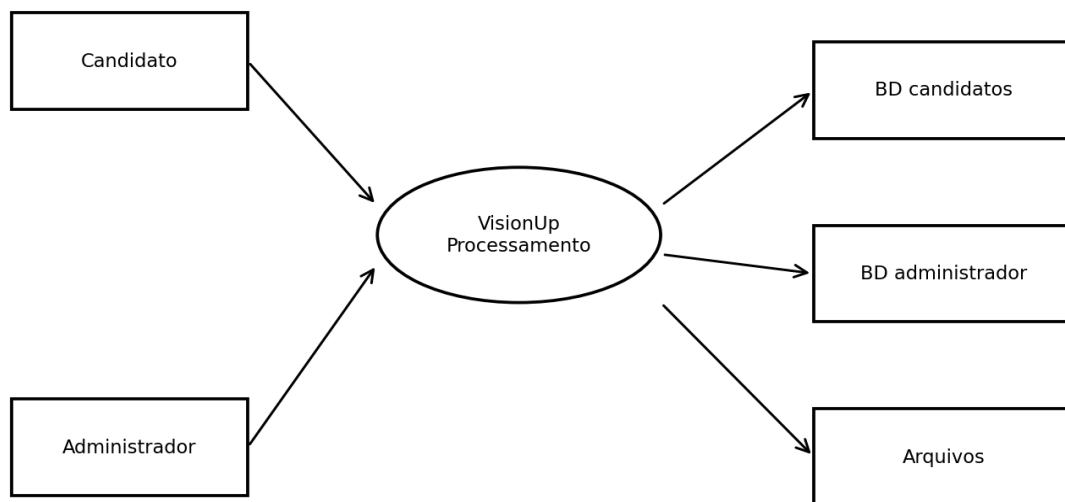
UC09 — Encerrar sessão: o sistema destrói a sessão e retorna à página inicial.

4. MODELAGEM DO SISTEMA

4.1 DFD (Diagrama de Fluxo de Dados)

O DFD simplificado representa o candidato e o administrador como entidades externas que interagem com o processamento do VisionUp. O processamento consulta e grava dados na base de candidatos e administrador e utiliza uma área de armazenamento para os arquivos enviados.

DFD — Visão de contexto simplificada

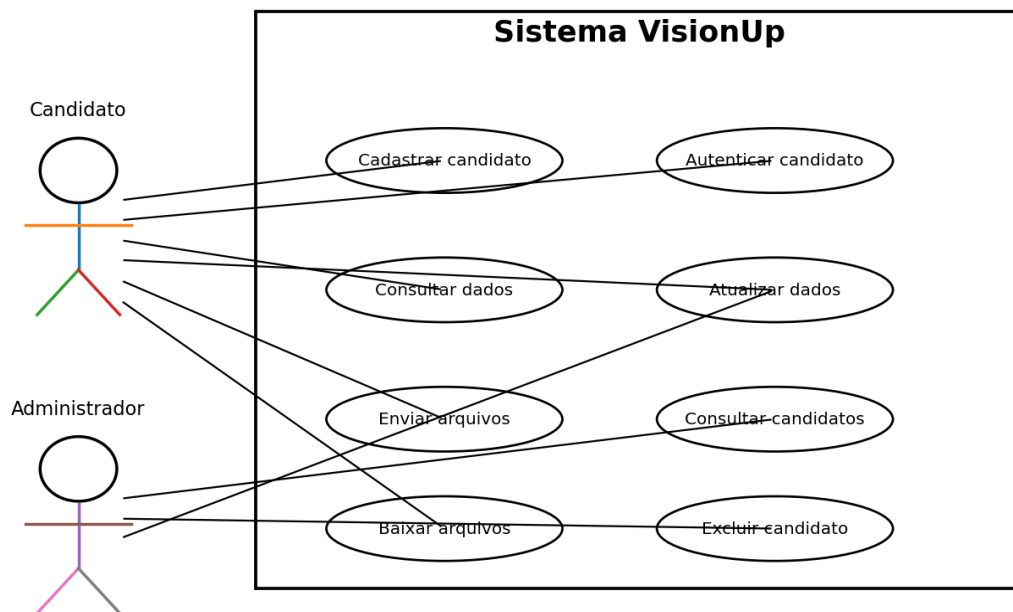


4.2 Diagramas UML

Os diagramas abaixo representam os principais elementos e fluxos do sistema a partir das funcionalidades identificadas no código-fonte.

4.2.1 Diagrama de Casos de Uso

O diagrama apresenta os atores Candidato e Administrador e suas principais interações com o sistema.

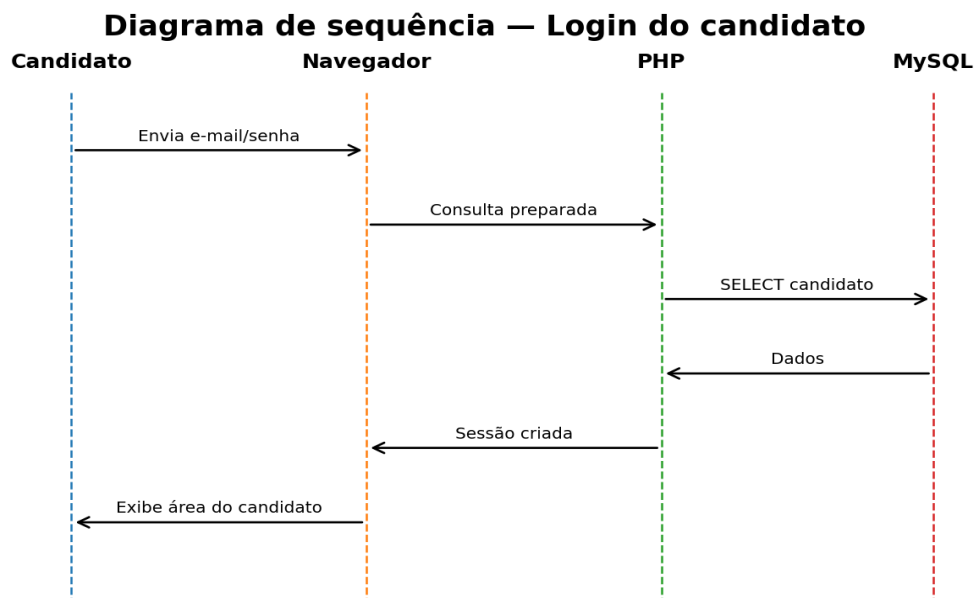


4.2.2 Diagrama de Classes

Em razão de o projeto ser predominantemente procedural em PHP, não existe uma camada de classes de domínio implementada. Para fins de modelagem, a estrutura de dados pode ser representada conceitualmente pelas entidades Candidato e Administrador, correspondentes às tabelas do banco.

4.2.3 Diagrama de Sequência

O fluxo apresentado corresponde à autenticação do candidato: entrada de credenciais, consulta preparada ao banco, retorno do registro, verificação da senha, criação da sessão e apresentação da área do candidato.



4.2.4 (Opcional) Diagrama de Atividades

O fluxo de cadastro pode ser representado como: iniciar → preencher formulário → validar campos → verificar e-mail → gerar hash → processar uploads → inserir candidato → informar sucesso ou erro.

4.2.5 (Opcional) Diagrama de Estados

Para o candidato, os estados principais podem ser descritos como: não autenticado → cadastro → autenticado → consulta/atualização → logout → não autenticado.

5. PROTOTIPAÇÃO

5.1 Protótipo no Figma

Não incluído nesta documentação, conforme orientação fornecida para a Avaliação A2. As interfaces foram implementadas diretamente em HTML/CSS/Bootstrap.

5.2 Descrição das telas (Documentação das interfaces)

Página inicial: apresenta as opções Administradores e Candidatos.

Login do candidato: recebe e-mail e senha, possui botão para mostrar/ocultar senha e opção de cadastro.

Cadastro do candidato: reúne nome, e-mail, telefone, senha, data de nascimento, endereço, dois trabalhos, currículo e LinkedIn. O CEP dispara consulta à API ViaCEP.

Área/dados do candidato: apresenta as informações cadastradas e permite atualizar os dados.

Login do administrador: recebe usuário e senha e possui controle visual para mostrar/ocultar senha.

Lista de candidatos: exibe os registros da tabela candidatos em formato de tabela, com links para documentos, LinkedIn e ação de exclusão.

6. ARQUITETURA DO SISTEMA

6.1 Visão geral da arquitetura

A arquitetura é web e segue uma separação simples entre interface, processamento no servidor, persistência de dados e armazenamento de arquivos. O navegador envia requisições HTTP para páginas PHP. Os scripts PHP realizam validações, autenticação, consultas e atualizações por meio do mysqli. Os dados estruturados ficam no banco MySQL/MariaDB e os arquivos enviados ficam na pasta curriculos/. O navegador também consulta a API ViaCEP diretamente por JavaScript para completar o endereço.

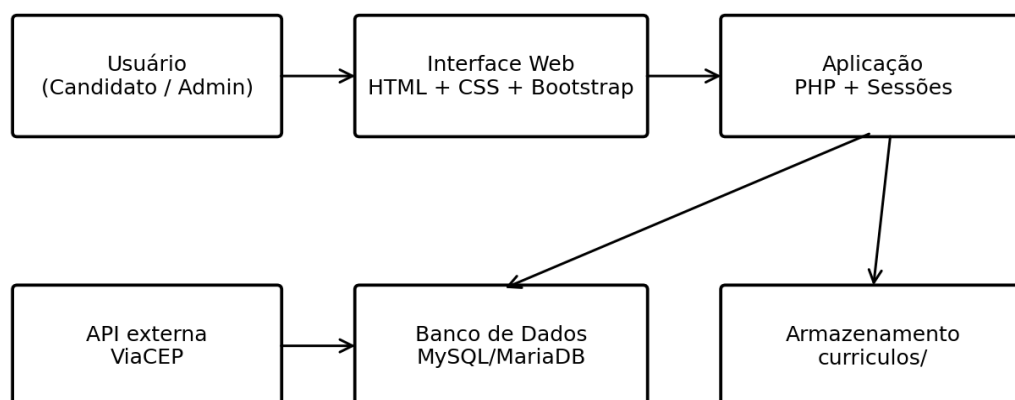
6.2 Tecnologias utilizadas

HTML5 — estrutura das páginas. CSS — estilos complementares. Bootstrap 5 — componentes e responsividade. JavaScript — interação no cliente e consulta ao ViaCEP. PHP — processamento no servidor e gerenciamento de sessões. MySQL/MariaDB — persistência dos dados. mysqli — conexão e operações de banco. ViaCEP — consulta de endereço por CEP. GitHub — hospedagem do repositório do projeto.

6.3 Diagrama de arquitetura

O diagrama a seguir apresenta a visão lógica dos principais componentes.

Arquitetura lógica do VisionUp



7. PROJETO DE BANCO DE DADOS

7.1 Modelo Conceitual

O modelo conceitual possui duas entidades principais: Candidato e Administrador. Candidato representa as pessoas que enviam suas informações e documentos. Administrador representa os usuários responsáveis pelo gerenciamento dos candidatos. Não foi identificada relação por chave estrangeira entre as duas entidades.

7.2 Modelo Lógico

Tabela candidatos: id_candidato, nome, dt_nasc, trabalho1, trabalho2, curriculo, linkedin, cep, logradouro, bairro, cidade, estado, email, senha e telefone.

Tabela administrador: id_adm, nome e senha.

7.3 Modelo Físico

Banco de dados: vu.

Tabela candidatos:

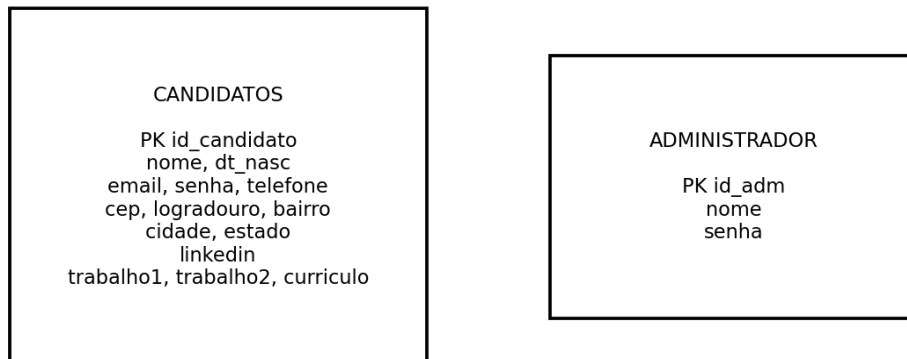
- id_candidato INT(11), chave primária, AUTO_INCREMENT, NOT NULL.
- nome VARCHAR(50), NOT NULL.
- dt_nasc DATE, NULL.
- trabalho1 VARCHAR(255), NULL.
- trabalho2 VARCHAR(255), NULL.
- curriculo VARCHAR(255), NULL.
- linkedin VARCHAR(255), NULL.
- cep VARCHAR(10), NULL.
- logradouro VARCHAR(50), NULL.
- bairro VARCHAR(50), NULL.
- cidade VARCHAR(50), NULL.
- estado VARCHAR(50), NULL.
- email VARCHAR(100), NOT NULL.
- senha VARCHAR(255), NOT NULL.
- telefone VARCHAR(20), NULL.

Tabela administrador:

- id_adm INT(11), chave primária.

- nome VARCHAR(50).
- senha VARCHAR(50).

Modelo físico simplificado do banco



8. DESENVOLVIMENTO / PROJETO FUNCIONAL

8.1 Código da aplicação (documentado)

O projeto possui páginas HTML para interfaces e scripts PHP para processamento. Entre os principais arquivos estão index.html (página inicial), cand.html (login do candidato), cand_cad.php (cadastro), cli_cad_pro.php (processamento do cadastro), logincand2.php (autenticação do candidato), alt_cand.php e salvar_cand.php (atualização), adm.html e loginadm.php (acesso administrativo), cons_cand.php (consulta administrativa), excluir_candidato.php (exclusão), logout.php (encerramento da sessão) e conexao.php (conexão com o banco).

O cadastro utiliza `mysqli_prepare()` e `mysqli_stmt_bind_param()`, reduzindo a exposição a SQL Injection no fluxo de candidatos. A senha do candidato é processada com `password_hash()`, e o login utiliza `password_verify()`. Os arquivos são processados por `$_FILES` e movidos com `move_uploaded_file()`.

8.2 Scripts SQL (documentados)

As operações SQL identificadas são principalmente SELECT, INSERT, UPDATE e DELETE. Exemplos funcionais: SELECT para autenticação e listagem; INSERT para cadastro de candidatos; UPDATE para manutenção cadastral; DELETE para exclusão administrativa. O projeto não depende de um arquivo SQL único dentro do ZIP; a estrutura foi informada a partir do banco utilizado.

8.3 Padrões de codificação utilizados

Foram observados nomes de arquivos e variáveis em português, uso de `session_start()` para sessões, `include_once()` para conexão, prepared statements nos fluxos de candidatos, tratamento de retorno com if/else, filtros básicos em entradas administrativas e organização visual por comentários em blocos. Como melhorias futuras, recomenda-se padronizar consultas preparadas em todos os módulos, separar configurações sensíveis do código e adotar uma arquitetura em camadas.

9. PLANO DE TESTES

9.1 Estratégia de testes

A estratégia adotada é baseada em testes funcionais manuais, verificando entradas válidas, entradas inválidas, autenticação, persistência, upload, consulta, atualização e exclusão. Também devem ser observados testes de interface em diferentes larguras de tela e testes de integração com a API ViaCEP.

9.2 Casos de teste

CT01 — Cadastro com dados obrigatórios válidos: esperado, candidato gravado com sucesso.
CT02 — Cadastro com e-mail já existente: esperado, sistema informa que o e-mail já está cadastrado.
CT03 — Cadastro sem nome/e-mail/senha: esperado, retorno ao formulário.
CT04 — Login de candidato com senha correta: esperado, sessão criada.
CT05 — Login de candidato com senha incorreta: esperado, acesso negado.
CT06 — CEP válido: esperado, preenchimento automático do endereço.
CT07 — CEP inválido/não encontrado: esperado, mensagem de CEP não encontrado.
CT08 — Upload de currículo/trabalhos: esperado, arquivos armazenados e caminhos gravados.
CT09 — Atualização de dados: esperado, registro alterado pelo id_candidato.
CT10 — Login administrativo válido: esperado, acesso à lista de candidatos.
CT11 — Consulta administrativa: esperado, candidatos listados.
CT12 — Download de arquivo: esperado, documento disponibilizado pelo link.
CT13 — Exclusão de candidato: esperado, registro removido.
CT14 — Logout: esperado, sessão destruída e retorno à página inicial.

9.3 Resultados dos testes

Com base na implementação fornecida, os fluxos principais estão implementados. O cadastro de candidato utiliza validação de campos básicos, verificação de e-mail, hash de senha, upload e INSERT. O login de candidato utiliza prepared statement e password_verify. A atualização e exclusão utilizam prepared statements. Recomenda-se, antes da entrega final, executar cada caso de teste em ambiente publicado e registrar evidências com prints, data e resultado observado.

10. CRONOGRAMA DO PROJETO

10.1 Etapas

1) levantamento de requisitos; 2) definição das telas e fluxo; 3) modelagem do banco; 4) desenvolvimento das interfaces; 5) implementação do cadastro; 6) autenticação; 7) upload e consulta de documentos; 8) módulo administrativo; 9) testes e correções; 10) publicação e documentação.

10.2 Prazos

Como não foram fornecidas datas oficiais de início e término, os prazos são apresentados como etapas sequenciais. A equipe pode preencher as datas reais de execução no quadro de acompanhamento do projeto.

10.3 Marcos (milestones)

M1 — Banco de dados criado. M2 — Interface inicial concluída. M3 — Cadastro de candidato funcionando. M4 — Login do candidato funcionando. M5 — Upload e consulta de documentos funcionando. M6 — Módulo administrativo funcionando. M7 — Testes finais. M8 — Publicação e entrega da documentação.

11. ORÇAMENTO

11.1 Custos estimados

Para o contexto acadêmico, os custos de desenvolvimento não foram definidos como contratação comercial. O sistema utiliza tecnologias de código aberto e um ambiente de hospedagem compartilhada. Eventuais custos externos dependem do provedor de hospedagem, domínio e necessidade de serviços adicionais.

11.2 Recursos necessários

Computador para desenvolvimento; editor de código; servidor local ou hospedagem PHP; MySQL/MariaDB; navegador web; conexão com a internet; conta/repositório GitHub; serviço ViaCEP para consulta de endereços.

12. IMPLANTAÇÃO E MANUTENÇÃO

12.1 Plano de implantação

1) criar o banco de dados vu; 2) criar as tabelas candidatos e administrador; 3) configurar os parâmetros de conexão no servidor; 4) enviar os arquivos PHP/HTML/CSS; 5) criar e garantir permissão da pasta curriculos/; 6) cadastrar o usuário administrador; 7) realizar testes de login, cadastro, upload, consulta e exclusão; 8) disponibilizar a URL pública.

12.2 Suporte e manutenção

A manutenção deve contemplar correções de erros, atualização do PHP e Bootstrap, revisão das regras de upload, cópias de segurança do banco e arquivos, acompanhamento da disponibilidade da API ViaCEP e revisão periódica de segurança.

Melhorias recomendadas: utilizar prepared statements também no login administrativo; aplicar password_hash/password_verify à tabela administrador; mover credenciais do banco para variáveis de ambiente ou configuração protegida; validar extensão, MIME type, tamanho e nome dos uploads no servidor; controlar melhor as permissões de acesso; adicionar proteção CSRF aos formulários; validar e escapar a saída HTML; restringir o acesso aos arquivos enviados quando houver necessidade de privacidade.

13. REFERÊNCIAS BIBLIOGRÁFICAS

13.1 Referências

PHP. Documentação oficial da linguagem PHP. <https://www.php.net/docs.php>

MySQL. Documentação oficial do MySQL. <https://dev.mysql.com/doc/>

Bootstrap. Documentação oficial do Bootstrap 5. <https://getbootstrap.com/docs/5.3/>

ViaCEP. Webservice para consulta de CEP. <https://viacep.com.br/>

GitHub. Plataforma de hospedagem e colaboração de código. <https://github.com/>

VisionUp. Sistema desenvolvido para a Avaliação A2. <https://visionup.rf.gd/index.html?i=1>

14. ANEXOS

14.1 Links úteis

Site publicado: <https://visionup.rf.gd/index.html?i=1>

Repositório/projeto GitHub: <https://github.com/pedrohctrezeleste/ProjetoRh/projects>

ViaCEP: <https://viacep.com.br/>

14.2 Protótipos

Não incluídos, conforme orientação recebida para a documentação.

14.3 Outros documentos

Podem ser anexados posteriormente: prints dos testes, estrutura do banco no phpMyAdmin, evidências de publicação, histórico de commits e demais materiais solicitados pela disciplina.

15. LINK DO REPOSITÓRIO

15.1 GitHub do projeto

O projeto está associado ao GitHub informado:

<https://github.com/pedrohctrezeleste/ProjetoRh/projects>

O repositório deve ser utilizado como evidência do desenvolvimento e como fonte para consulta do código da aplicação.

ANEXO A — ESTRUTURA DETALHADA DO BANCO

Tabela	Campo	Tipo	Observação
candidatos	id_candidato	INT(11)	PK; AUTO_INCREMENT; NOT NULL
candidatos	nome	VARCHAR(50)	NOT NULL
candidatos	dt_nasc	DATE	NULL
candidatos	trabalho1	VARCHAR(255)	NULL; caminho do arquivo
candidatos	trabalho2	VARCHAR(255)	NULL; caminho do arquivo
candidatos	curriculo	VARCHAR(255)	NULL; caminho do arquivo
candidatos	linkedin	VARCHAR(255)	NULL
candidatos	cep	VARCHAR(10)	NULL
candidatos	logradouro	VARCHAR(50)	NULL
candidatos	bairro	VARCHAR(50)	NULL
candidatos	cidade	VARCHAR(50)	NULL
candidatos	estado	VARCHAR(50)	NULL
candidatos	email	VARCHAR(100)	NOT NULL
candidatos	senha	VARCHAR(255)	NOT NULL; hash do candidato
candidatos	telefone	VARCHAR(20)	NULL
administrador	id_adm	INT(11)	PK
administrador	nome	VARCHAR(50)	
administrador	senha	VARCHAR(50)	

Observação de segurança: o arquivo conexao.php do pacote original contém credenciais do ambiente publicado. Elas não foram reproduzidas neste documento. Recomenda-se substituir credenciais expostas e utilizar configuração segura.